

SPRING BOOT Mastery

Kurumsal Uygulama Geliřtirme Eđitimi



Neominal Akademi
2026

Dokümanın Amacı

Bu teorik konu anlatım dökümanı, Neominal Akademi tarafından sunulan "**Spring Framework & Spring Boot ile Kurumsal Uygulama Geliştirme**" eğitimine katılacak profesyoneller için hazırlanmıştır.

Amaç, katılımcıların 3 günlük eğitim süresince işlenecek teknik konuları, hem eğitim öncesinde ön bilgi olarak hem de eğitim sonrasında başvuru kaynağı olarak kullanabilecekleri kapsamlı ve yapılandırılmış bir kaynakla desteklemektir.

Hakkında

Bu doküman, Spring Framework'ün temellerinden başlayarak Spring Boot ile kurumsal düzeyde REST tabanlı, veritabanı destekli, güvenli ve production'a hazır bir Java uygulamasının nasıl inşa edildiğini teorik düzeyde ele alır. İçerik, Clean Architecture / N-Tier katmanlı mimari yaklaşımı temel alınarak hazırlanmıştır.

Eğitim Genel Bakış

Özellik	Detay
Süre	3 gün (12 saat toplam - günde 4 saat)
Seviye	Orta / İleri seviye - Kurumsal
Format	Teorik anlatım + Kod örnekleri + Best Practice paylaşımları
Modüller	3 ana modül (Spring Temelleri ve Mimari, Uygulama Katmanı ve Veri Erişimi, Kalite / Güvenlik / Operasyon)
Ön Koşul	Temel Java (OOP, Collections), temel SQL ve HTTP/REST kavramlarına aşinalık

Ön Koşul Bilgi Kontrolü

Bu eğitim **orta/ileri seviye** bir programdır ve katılımcıların belirli temel bilgilere sahip olması beklenmektedir. Aşağıdaki listeyi kontrol ederek hazırlık seviyenizi değerlendirebilirsiniz.

Bilgi Seviyesi Self-Assessment

Aşağıdaki maddelere "Evet" diyebiliyor musunuz? Kontrol edin:

Java Temelleri

- ☐ Nesne yönelimli programlama (OOP) kavramlarını (kalıtım, polimorfizm, encapsulation) biliyorum
- ☐ Interface ve Abstract Class farkını anlıyorum
- ☐ Collections API'sini (List, Map, Set) rahatça kullanabiliyorum
- ☐ Lambda ifadeleri ve Stream API kullanabiliyorum
- ☐ Exception handling (try-catch, checked/unchecked) yapabiliyorum
- ☐ Maven veya Gradle ile proje bağımlılığı yönetebiliyorum

Web ve HTTP Temelleri

- ☐ HTTP metodlarını (GET, POST, PUT, DELETE) ve durum kodlarını biliyorum
- ☐ JSON formatını okuyup yazabiliyorum

- ☐ REST API kavramını duydum
- ☐ Postman veya benzeri bir araçla API test edebiliyorum

Veritabanı Temelleri

- ☐ Temel SQL sorgularını (SELECT, INSERT, UPDATE, DELETE) yazabiliyorum
- ☐ Primary Key / Foreign Key kavramlarını biliyorum
- ☐ JOIN mantığını anlıyorum
- ☐ İlişkisel veritabanı (PostgreSQL/MySQL) ile çalıştım

Genel Yazılım Geliştirme

- ☐ Git ile versiyon kontrolü yapabiliyorum
- ☐ IDE (IntelliJ IDEA / Eclipse) kullanımına aşinayım
- ☐ Katmanlı mimari (layered architecture) kavramını duydum
- ☐ Birim test (unit test) kavramına aşinayım

Eksikleriniz Varsa Ne Yapmalısınız?

Yukarıdaki maddelerden herhangi birinde eksiklik hissediyorsanız, aşağıdaki kaynaklarla kendinizi güçlendirebilirsiniz:

Java Temelleri

- Oracle Java Tutorials: <https://docs.oracle.com/javase/tutorial/>
- Baeldung - Java: <https://www.baeldung.com/java-tutorial>

Spring / Spring Boot

- Spring Resmi Dökümantasyon: <https://docs.spring.io/spring-boot/>
- Baeldung - Spring: <https://www.baeldung.com/spring-tutorial>

SQL / PostgreSQL

- PostgreSQL Resmi Tutorial: <https://www.postgresqltutorial.com/>
- W3Schools SQL: <https://www.w3schools.com/sql/>

Video Kaynaklar

- Udemy - Spring Boot ile Mikroservis / REST API Geliştirme:

MODÜL 1

SPRING TEMELLERİ VE MİMARİ

Framework'ten Boot'a Geçiş

IoC • Dependency Injection • Auto-Configuration • Starter

İçindekiler

1. Spring Framework ve Spring Boot Farkları
2. Spring Boot Mimarisi
3. Spring Boot ile Ne Tür Projeler Yazılır
4. Spring IoC & Dependency Injection

1. Spring Framework ve Spring Boot Farkları

Spring Framework, 2003 yılında J2EE'nin (Java EE) ağır ve karmaşık geliştirme modeline alternatif olarak ortaya çıkmış; Inversion of Control (IoC) ve Dependency Injection (DI) prensipleriyle Java kurumsal uygulama geliştirmede devrim yaratmış bir framework'tür. Ancak zamanla Spring'in kendisi de büyük ölçüde XML tabanlı konfigürasyon, çok sayıda bağımlılık yönetimi ve manuel sunucu kurulumu nedeniyle karmaşıklaşmıştır.

1.1. Spring Framework'ün Getirdiği Problemler ve Çözümler

Spring Framework'ün temel katkısı, nesneler arası bağımlılıkların kod içinde new anahtar kelimesiyle sıkı biçimde oluşturulması yerine, bir **IoC Container** tarafından yönetilmesidir. Bu sayede test edilebilirlik, gevşek bağıllık (loose coupling) ve modülerlik sağlanmıştır.

- **Sıkı Bağımlılık Problemi:** Nesnelerin birbirini doğrudan oluşturması yerine container üzerinden enjekte edilmesi
- **Boilerplate Kod:** AOP ile transaction, loglama gibi tekrar eden kodların merkezi noktadan yönetilmesi
- **Test Edilebilirlik:** Bağımlılıkların mock'lanabilir hale gelmesi

1.2. Spring Boot'un Felsefesi: Convention over Configuration

Spring Boot, 2014 yılında Pivotal tarafından, Spring Framework'ün gücünü korurken kurulum ve konfigürasyon yükünü ortadan kaldırmak amacıyla geliştirilmiştir. Temel felsefesi "**Convention over Configuration**" (Konfigürasyon yerine Kural/Gelenek) ilkesidir: geliştirici aksini belirtmediği sürece, framework makul varsayılan değerlerle (sensible defaults) çalışır.

- **Auto-Configuration:** Classpath'teki kütüphanelere bakarak gerekli bean'leri otomatik oluşturur
- **Starter Dependency:** `spring-boot-starter-web` gibi tek bir bağımlılık, ilgili tüm alt kütüphaneleri birlikte getirir
- **Embedded Server:** Tomcat/Jetty/Undertow uygulamanın içine gömülür, ayrı sunucu kurulumuna gerek kalmaz
- **Production-Ready Özellikler:** Actuator ile health-check, metrik ve monitoring hazır gelir

1.3. Karşılaştırma Tablosu

Özellik	Spring Framework	Spring Boot
Konfigürasyon	Genellikle XML veya manuel <code>@Configuration</code> sınıfları gerekir	Auto-Configuration ile otomatik, gerektiğinde override edilebilir
Sunucu Kurulumu	Harici WAR deploy (Tomcat/JBoss kurulumu gerekir)	Embedded server, <code>java -jar</code> ile doğrudan çalışır
Bağımlılık Yönetimi	Her kütüphane tek tek eklenir ve versiyon uyumu manuel takip edilir	Starter POM'lar ile toplu ve uyumlu versiyon yönetimi
Öğrenme Eğrisi	Daha dik, çok sayıda kavramın manuel öğrenilmesi gerekir	Daha düşük, hızlı prototipleme mümkündür
Kullanım Amacı	Framework'ün alt seviye bileşenlerini esnek biçimde birleştirmek	Hızlı, production-ready mikroservis ve web uygulaması geliştirmek

BİLGİ — Önemli Nokta

Spring Boot, Spring Framework'ün **yerine geçen** bir teknoloji değildir; onun üzerine inşa edilmiş, kurulum ve konfigürasyon karmaşıklığını azaltan bir **dağıtım/başlangıç modelidir**. Spring Boot uygulaması içinde de saf Spring IoC, AOP ve MVC bileşenleri çalışmaya devam eder.

1.4. Ne Zaman Hangisi Tercih Edilmeli?

Günümüzde yeni başlayan neredeyse tüm kurumsal Java projelerinde Spring Boot tercih edilmektedir. Saf Spring Framework kullanımı genellikle çok özel, yüksek düzeyde özelleştirilmiş legacy sistem entegrasyonları veya framework'ün kendisinin geliştirilmesi gibi nadir senaryolarla sınırlıdır.

2. Spring Boot Mimarisi

Spring Boot mimarisini anlamak, bir uygulamanın main metodundan itibaren nasıl ayağa kalktığını ve hangi mekanizmaların devreye girdiğini bilmeyi gerektirir.

2.1. @SpringBootApplication Anotasyonunun Anatomisi

Her Spring Boot uygulamasının giriş noktasında bulunan @SpringBootApplication anotasyonu aslında üç anotasyonun birleşimidir:

Anotasyon	Görevi
@Configuration	Sınıfın bean tanımları içeren bir konfigürasyon kaynağı olduğunu belirtir
@EnableAutoConfiguration	Classpath'e göre otomatik bean konfigürasyonunu tetikler
@ComponentScan	Paket altındaki @Component, @Service, @Repository, @Controller sınıflarını tarar

```
@SpringBootApplication
public class SpringApplication {
    public static void main(String[] args) {
        SpringApplication.run(SiparisYonetimApplication.class, args);
    }
}
```

2.2. Uygulama Başlatma Yaşam Döngüsü

SpringApplication.run() çağrıldığında sırasıyla şu adımlar gerçekleşir:

- ApplicationContext Oluşturma:** Uygulama türüne göre (Web/Reactive/Standalone) uygun context seçilir
- Environment Hazırlama:** application.properties/yml, ortam değişkenleri ve komut satırı argümanları birleştirilir
- Auto-Configuration Çalıştırma:** Classpath taranır, koşullu (@ConditionalOnClass vb.) konfigürasyonlar devreye girer
- Bean'lerin Oluşturulması:** IoC Container tüm bean'leri instantiate eder ve bağımlılıkları enjekte eder
- Embedded Server Başlatma:** Web uygulamasıysa Tomcat/Jetty gömülü sunucu ayağa kalkar
- ApplicationRunner/CommandLineRunner Çalıştırma:** Uygulama açılışında çalışması gereken kod parçaları tetiklenir

2.3. application.properties / application.yml

Konfigürasyon değerleri merkezi olarak application.properties veya application.yml dosyalarında tutulur. YML formatı, hiyerarşik yapı sunması nedeniyle kurumsal projelerde daha sık tercih edilir.

```
spring:
  application:
    name: siparis-servisi
  datasource:
    url: jdbc:postgresql://localhost:5432/siparisdb
```

```
username: postgres
password: secret
server:
port: 8080
```

2.4. Profil Yönetimi (Profiles)

Farklı ortamlar (dev, test, prod) için farklı konfigürasyon setleri kullanmak amacıyla Spring Profiles mekanizması kullanılır. `application-dev.yml`, `application-prod.yml` gibi dosyalar oluşturulur ve aktif profil `spring.profiles.active` ile seçilir.

- **dev:** Yerel geliştirme — detaylı loglama, H2/in-memory veritabanı seçenekleri
- **test:** Otomatik test ortamı — izole test veritabanı
- **prod:** Canlı ortam — performans odaklı, minimal loglama, gerçek veritabanı bağlantısı

2.5. Auto-Configuration Mekanizması

Spring Boot, `spring-boot-autoconfigure` kütüphanesi içindeki `AutoConfiguration.imports` dosyasında listelenen konfigürasyon sınıflarını, classpath'teki mevcut kütüphanelere ve tanımlı bean'lere göre koşullu olarak devreye sokar. Örneğin `spring-boot-starter-data-jpa` bağımlılığı eklendiğinde, `DataSourceAutoConfiguration` ve `HibernateJpaAutoConfiguration` otomatik olarak etkinleşir.

NOT — Debug İpucu

`--debug` parametresiyle uygulama çalıştırıldığında, konsola hangi auto-configuration sınıflarının etkin/pasif olduğunu gösteren bir **Auto-Configuration Report** yazdırılır. Bu, beklenmeyen davranışları teşhis etmek için değerlidir.

3. Spring Boot ile Ne Tür Projeler Yazılır

Spring Boot'un modüler starter yapısı, aynı temel framework üzerinde birbirinden çok farklı uygulama tiplerinin geliştirilmesine olanak tanır.

3.1. Proje Tipleri

Proje Tipi	Açıklama	Tipik Starter
Monolitik Web Uygulaması	Tüm iş mantığının tek bir deploy edilebilir birim içinde bulunduğu klasik uygulama	spring-boot-starter-web, thymeleaf
RESTful Backend API	Frontend'den (React/Angular/mobil) bağımsız, JSON tabanlı servis katmanı	spring-boot-starter-web
Mikroservis	Bağımsız deploy edilebilen, tek sorumluluğa sahip küçük servisler	spring-cloud-starter-*, Eureka, Config Server
Batch / Zamanlanmış İş	Büyük veri kümeleri üzerinde periyodik toplu işlem yapan uygulamalar	spring-boot-starter-batch, @Scheduled
Mesajlaşma Tabanlı Sistem	Event-driven mimaride asenkron mesaj işleme	spring-kafka, spring-boot-starter-amqp
Reaktif Uygulama	Yüksek eşzamanlılık gerektiren, non-blocking I/O tabanlı sistemler	spring-boot-starter-webflux

3.2. Mikroservis Mimarisine Kısa Bakış

Spring Boot, tek başına bir mikroservis çatısı değildir; ancak **Spring Cloud** ekosistemiyle birleştğinde servis kaydı (Eureka), merkezi konfigürasyon (Config Server), API Gateway ve devre kesici (Circuit Breaker/Resilience4j) gibi mikroservis gereksinimlerini karşılayabilir hale gelir. Bu eğitimde odak, tekil bir Spring Boot servisinin sağlam biçimde inşa edilmesidir; mikroservis orkestrasyonu kapsam dışıdır.

3.3. Ne Zaman Spring Boot Tercih Edilmeli?

- Hızlı MVP/prototip geliştirme gerektiğinde
- Kurumsal, uzun ömürlü ve bakımı sürdürülebilir backend sistemleri kurulurken
- Ekip standardizasyonu ve geniş topluluk desteği önem taşıdığı anda
- Cloud-native, container tabanlı (Docker/Kubernetes) dağıtım hedeflendiğinde

4. Spring IoC & Dependency Injection

Spring'in kalbinde yatan kavram **Inversion of Control (IoC)**'dir: nesnelerin yaşam döngüsünü ve bağımlılıklarını yönetme sorumluluğu, geliştiriciden alınıp container'a devredilir.

4.1. IoC Container Kavramı

Spring'de IoC, `ApplicationContext` arayüzü üzerinden somutlaşır. `BeanFactory`, en temel container arayüzüdür ve lazy-loading yapar; `ApplicationContext` ise onu genişletir, ek olarak event yayınlama, i18n desteği ve eager (istekli) bean yükleme gibi kurumsal özellikler sunar. Modern Spring Boot uygulamalarında neredeyse her zaman `ApplicationContext` kullanılır.

4.2. Bean Yaşam Döngüsü

7. **Instantiation:** Bean'in constructor'ı çağrılarak nesne oluşturulur
8. **Populate Properties:** Bağımlılıklar (diğer bean'ler) enjekte edilir
9. **Aware Interface Çağrılar:** `BeanNameAware`, `ApplicationContextAware` gibi arayüzler varsa tetiklenir
10. **BeanPostProcessor (Before):** Init öncesi ara işlemler
11. **@PostConstruct / afterPropertiesSet():** Bean hazır olduğunda çalışacak özel başlatma kodu
12. **Bean Kullanıma Hazır:** Container tarafından enjekte edilmeye hazırdır
13. **@PreDestroy:** Container kapatılırken temizlik (cleanup) kodu çalışır

4.3. Dependency Injection Türleri

DI Türü	Açıklama	Tavsiye
Constructor Injection	Bağımlılıklar constructor parametresi olarak alınır, final alanlarla immutability sağlanır	Önerilen yöntem — zorunlu bağımlılıklar için
Setter Injection	Bağımlılık, setter metodu ile sonradan atanır	Opsiyonel bağımlılıklar için kullanılabilir
Field Injection	@Autowired doğrudan alan (field) üzerine yazılır	Test edilebilirliği zayıflattığı için önerilmez

```
@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final StockService stockService;

    // Constructor Injection - tek constructor varsa @Autowired zorunlu değildir
    public OrderService(OrderRepository orderRepository, StockService stockService) {
        this.orderRepository = orderRepository;
        this.stockService = stockService;
    }
}
```

Constructor Injection kullanmak, bağımlılıkların **zorunlu ve değişmez (immutable)** olmasını garanti eder; ayrıca birim testlerde mock nesnelerin kolayca verilmesine imkân tanır. Kurumsal projelerde standart yaklaşım budur.

4.4. Stereotype Anotasyonları

Anotasyon	Kullanım Amacı
@Component	Genel amaçlı, herhangi bir Spring-managed bean için temel anotasyon
@Service	İş mantığı (business logic) katmanındaki sınıflar için semantik işaretleyici
@Repository	Veri erişim katmanı; ek olarak veritabanı istisnalarını Spring'in <code>DataAccessException</code> hiyerarşisine çevirir
@Controller / @RestController	Web/HTTP isteklerini karşılayan sınıflar için kullanılır

4.5. @Autowired, @Qualifier ve @Primary

Aynı arayüzün birden fazla implementasyonu olduğunda, Spring hangi bean'in enjekte edileceğini belirleyemez ve `NoUniqueBeanDefinitionException` fırlatır. Bu durumda `@Qualifier` ile spesifik bean adı belirtilir, ya da `@Primary` ile bir implementasyon varsayılan olarak işaretlenir.

4.6. Bean Scope'ları

Scope	Açıklama
singleton (varsayılan)	Container başına tek bir bean örneği oluşturulur ve paylaşılır
prototype	Her <code>getBean()</code> çağrısında yeni bir örnek oluşturulur
request	Her HTTP isteği için ayrı bir bean örneği (web uygulamalarında)
session	Her HTTP session için ayrı bir bean örneği

4.7. Circular Dependency (Dairesel Bağımlılık) Problemi

İki bean birbirini constructor üzerinden enjekte etmeye çalıştığında, Spring bean'i oluştururken sonsuz döngüye girer ve `BeanCurrentlyInCreationException` fırlatılır. Çözüm genellikle mimari düzeydedir: **tasarım kokusu (design smell)** olarak ele alınmalı, sınıflardan biri parçalanmalı veya ortak bir üçüncü servise çıkarılmalıdır. Zorunlu hallerde `@Lazy` anotasyonu geçici çözüm sağlar.

MODÜL 2

UYGULAMA KATMANI VE VERİ ERİŞİMİ

AOP'den Katmanlı Mimariye

AOP • REST • JPA • Repository • N-Tier

İçindekiler

- 5. Spring AOP ve Annotation'lar
- 6. Spring Uygulamalarında RESTful Servis Mimarisi
- 7. Spring Data JPA & PostgreSQL & Repository
- 8. Clean Architecture ve Spring Boot N-Tier Mimari

5. Spring AOP ve Annotation'lar

Aspect Oriented Programming (AOP), bir uygulamanın birden fazla katmanını kesen (cross-cutting concern) loglama, transaction yönetimi, güvenlik ve performans ölçümü gibi sorumlulukları, iş mantığı kodundan ayırarak merkezi bir noktada yönetmeyi sağlayan programlama paradigmasıdır.

5.1. Temel AOP Kavramları

Kavram	Açıklama
Aspect	Cross-cutting bir sorumluluğu kapsülleyen modül (örn. LoggingAspect)
Advice	Aspect'in belirli bir join point'te yapacağı iş (ne zaman, ne yapılacağı)
Join Point	Kodun çalışma anındaki bir noktası; Spring AOP'de her zaman bir metod çağrısıdır
Pointcut	Hangi join point'lerin advice ile eşleştirileceğini tanımlayan ifade
Weaving	Aspect'in hedef koda bağlanma işlemi; Spring AOP'de proxy tabanlı, runtime'da yapılır

5.2. Advice Türleri

Advice	Ne Zaman Çalışır
@Before	Hedef metod çalışmadan önce
@After	Hedef metod çalıştıktan sonra (sonuç ne olursa olsun)
@AfterReturning	Hedef metod başarıyla değer döndürdükten sonra
@AfterThrowing	Hedef metod bir istisna fırlattığında
@Around	Hedef metodun öncesinde ve sonrasında; metodun çalışıp çalışmayacağını da kontrol edebilir

5.3. Örnek: Loglama Aspect'i

```
@Aspect
@Component
public class LoggingAspect {

    private static final Logger log = LoggerFactory.getLogger(LoggingAspect.class);

    @Around("execution(* com.firma.siparis.service.*(..))")
    public Object logExecutionTime(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = pjp.proceed(); // hedef metodu çalıştır
        long sure = System.currentTimeMillis() - start;
        log.info("{} metodu {} ms sürdü", pjp.getSignature(), sure);
        return result;
    }
}
```

AOP; performans ölçümü, audit-log tutma, method-level güvenlik kontrolü ve retry mekanizmaları gibi senaryolarda, ilgili kodu her metoda tek tek yazmak yerine merkezi ve tekrar kullanılabilir biçimde uygulamayı sağlar.

5.4. Spring'in Diğer Yaygın Annotation'ları

Anotasyon	Açıklama
@Transactional	Bir metodu veritabanı transaction sınırı içine alır; AOP proxy mekanizmasıyla çalışır
@Value	application.yml içindeki bir değeri doğrudan alana enjekte eder
@ConfigurationProperties	Bir konfigürasyon bloğunu type-safe biçimde bir POJO'ya bağlar

6. Spring Uygulamalarında RESTful Servis Mimarisi

REST (Representational State Transfer), kaynak (resource) tabanlı, stateless (durumsuz) bir mimari stildir. Spring MVC, bu prensipleri @RestController üzerinden HTTP protokolüyle doğrudan eşleyerek uygular.

6.1. REST Prensipleri

- **Statelessness:** Sunucu istekler arasında client durumu tutmaz, her istek kendi içinde yeterli bilgiyi taşır
- **Resource-Based URI:** /orders/42 gibi kaynak odaklı, fiil içermeyen adresleme
- **HTTP Metodları ile Eylem:** GET (okuma), POST (oluşturma), PUT/PATCH (güncelleme), DELETE (silme)
- **Temsil (Representation):** Kaynağın JSON/XML gibi bir formatta istemciye sunulması

6.2. Controller Anotasyonları

Anotasyon	Görevi
@RestController	@Controller + @ResponseBody; dönen değerlerin doğrudan HTTP body'sine JSON olarak yazılmasını sağlar
@RequestMapping	Sınıf/metod seviyesinde temel URL eşlemesi
@GetMapping / @PostMapping / @PutMapping / @DeleteMapping	İlgili HTTP metoduna özel kısayol anotasyonları
@PathVariable	URL şablonundaki dinamik segmenti (/orders/{id}) parametreye bağlar
@RequestParam	Sorgu string'i (?status=active) parametrelerini bağlar
@RequestBody	HTTP isteğinin gövdesindeki JSON'ı Java nesnesine dönüştürür

6.3. Örnek Controller

```
@RestController
@RequestMapping("/api/v1/orders")
public class OrderController {

    private final OrderService orderService;

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<OrderDto> getById(@PathVariable Long id) {
        OrderDto response = orderService.getirById(id);
        return ResponseEntity.ok(response);
    }

    @PostMapping
    public ResponseEntity<OrderDto> create(@Valid @RequestBody OrderCreatedDto request) {
        OrderDto response = orderService.olustur(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }
}
```

```
}  
}
```

6.4. ResponseEntity ve HTTP Status Kodları

Kod	Anlamı	Tipik Kullanım
200 OK	Başarılı	GET, PUT başarılı sonuç
201 Created	Kaynak oluşturuldu	POST sonrası yeni kayıt
204 No Content	İçerik yok	DELETE sonrası
400 Bad Request	İstemci hatası	Validation hatası
401 / 403	Yetkisiz / Yasak	Authentication / Authorization hataları
404 Not Found	Kaynak bulunamadı	Var olmayan ID sorgusu
500 Internal Server Error	Sunucu hatası	Beklenmeyen istisna

6.5. DTO Kullanımının Önemi

Entity sınıflarının doğrudan API üzerinden dışarı sunulması, veritabanı yapısının istemciye sızmasına, gereksiz alanların ifşa olmasına ve JPA'nın lazy-loading kaynaklı LazyInitializationException hatalarına yol açabilir. Bu nedenle her katman kendi **DTO (Data Transfer Object)** sınıflarını kullanmalı, Entity ↔ DTO dönüşümü mapper sınıfları (veya MapStruct gibi kütüphaneler) ile yapılmalıdır.

6.6. API Versiyonlama

- **URI Versiyonlama:** /api/v1/orders — en yaygın ve anlaşılır yöntem
- **Header Versiyonlama:** Accept: application/vnd.firma.v2+json
- **Query Parametre:** /orders?version=2 — daha az tercih edilir

7. Spring Data JPA & PostgreSQL & Repository

Spring Data JPA, ORM (Object-Relational Mapping) katmanının Hibernate implementasyonu üzerine inşa edilmiş, tekrarlayan veri erişim kodunu (boilerplate) ortadan kaldıran bir soyutlama katmanıdır.

7.1. ORM ve Hibernate'in Rolü

JPA (Jakarta Persistence API) bir **spesifikasyondur**; Hibernate ise bu spesifikasyonun en yaygın implementasyonudur. Java nesnelerini (Entity) ilişkisel veritabanı tablolarına eşler, SQL üretimini ve nesne-tablo dönüşümünü otomatikleştirir.

7.2. Entity Mapping

```
@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "order_no", nullable = false, unique = true)
    private String orderNo;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "customer_id")
    private Customer customer;

    @OneToMany(mappedBy = "items", cascade = CascadeType.ALL)
    private List<OrderItem> items = new ArrayList<>();
}
```

7.3. PostgreSQL Bağlantı Yapılandırması

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/siparisdb
    username: postgres
    password: secret
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: validate # prod ortamında update/create asla kullanılmaz
      show-sql: true
    properties:
      hibernate:
        format_sql: true
```

DİKKAT — Dikkat

ddl-auto: update veya create seçenekleri **yalnızca geliştirme ortamında** kullanılmalıdır. Production ortamında şema değişiklikleri Flyway veya Liquibase gibi migration araçlarıyla kontrollü biçimde yönetilmelidir.

7.4. Repository Katmanı

Arayüz	Sağladığı Özellikler
CrudRepository<T, ID>	Temel CRUD operasyonları (save, findById, delete, findAll)
PagingAndSortingRepository<T, ID>	CrudRepository + sayfalama ve sıralama desteği
JpaRepository<T, ID>	Yukarıdakilerin tümü + JPA'ya özgü ek metodlar (flush, batch delete)

```
public interface OrderRepository extends JpaRepository<Order, Long> {

    // Query Method - metod isminden otomatik SQL üretimi
    List<Order> findByCustomerIdAndOrderStatus(Long customerId, OrderStatus status);

    // JPQL ile özel sorgu
    @Query("SELECT s FROM Orders s WHERE s.total > :total")
    List<Siparis> tutariBuyukOlanlar(@Param("total") BigDecimal total);

    // Native SQL sorgusu
    @Query(value = "SELECT * FROM orders WHERE orderNumber = :no", nativeQuery =
true)
    Optional<Siparis> findBySiparisNoNative(@Param("no") String no);
}
```

7.5. Query Method İsimlendirme Kuralları

Spring Data JPA, findById, existsBy, countBy, deleteBy gibi öneklerle başlayan ve entity alan adlarını içeren metod isimlerinden otomatik olarak SQL sorgusu üretir. And, Or, Between, GreaterThan, OrderBy gibi anahtar kelimeler desteklenir.

7.6. Pagination & Sorting

```
Pageable pageable = PageRequest.of(0, 20, Sort.by("createdAt").descending());
Page<Order> sayfa = orderRepository.findAll(pageable);
```

7.7. Transaction Yönetimi Temelleri

@Transactional anotasyonu, bir metod bloğunun tek bir veritabanı transaction'ı içinde çalışmasını garanti eder; metod içinde bir istisna oluşursa (varsayılan olarak unchecked exception) tüm işlemler rollback edilir. Servis (business logic) katmanında kullanılması, controller katmanında kullanılmaması önerilir.

8. Clean Architecture ve Spring Boot N-Tier Mimari

Sürdürülebilir ve test edilebilir bir Spring Boot uygulaması, sorumlulukları net biçimde ayrılmış katmanlar üzerine kurulmalıdır. Bu bölümde klasik N-Tier (katmanlı) yaklaşım ile Clean Architecture prensipleri birlikte ele alınmaktadır.

8.1. Klasik N-Tier Katman Yapısı

Katman	İçerdiği Bileşenler	Sorumluluk
Controller (Presentation)	@RestController	HTTP isteğini karşılar, DTO'ya çevirir, servis katmanını çağırır
Service (Business Logic)	@Service, arayüzler	İş kurallarını uygular, transaction sınırlarını belirler
Repository (Data Access)	@Repository, JpaRepository	Veritabanı ile iletişim kurar
Entity / Domain	@Entity	Veritabanı tablosuna karşılık gelen domain modeli

Bağımlılık yönü her zaman **dıştan içe** doğrudur: Controller → Service → Repository → Entity. Bir üst katman bir alt katmanı bilir, tersi asla olmamalıdır.

8.2. DTO / Mapper Kullanımı

Katmanlar arası veri taşımada Entity nesnelerinin doğrudan kullanılması yerine her katman sınırında (özellikle Controller-Service arası) DTO kullanılmalıdır. Dönüşüm işlemleri elle yazılan mapper sınıflarıyla veya derleme zamanında kod üreten **MapStruct** kütüphanesiyle yapılabilir.

8.3. Clean Architecture Prensipleri

Robert C. Martin tarafından tanımlanan Clean Architecture, iş mantığının (domain) framework, veritabanı ve UI detaylarından **bağımsız** olmasını savunur. Spring Boot projelerinde bu, iş kurallarının doğrudan JPA Entity'lerine veya Spring anotasyonlarına bağımlı olmaması, arayüzler (port) üzerinden soyutlanması anlamına gelir.

- **Bağımsızlık:** Domain katmanı hiçbir framework'e bağımlı olmamalı
- **Test Edilebilirlik:** İş kuralları, veritabanı veya web sunucusu olmadan test edilebilmeli
- **Framework'ten Bağımsızlık:** Spring, sadece dış katmanlarda bir detay olarak kalmalı

8.4. Paket Organizasyonu Stratejileri

Strateji	Açıklama
Layered (Katman Bazlı)	controller/, service/, repository/, entity/ — küçük/orta projelerde yaygın
Feature-Based (Özellik Bazlı)	order/, customer/, stock/ gibi her iş alanı kendi paketinde tüm katmanlarıyla birlikte bulunur — büyük projelerde ve mikroservice geçişte tercih edilir

8.5. Servis Arayüzlerinin Önemi

Servis katmanında somut sınıf yerine arayüz (interface) tanımlamak; implementasyonun değiştirilebilir olmasını, test sırasında mock nesnelerin kolayca enjekte edilmesini ve Clean Architecture'daki port-adapter mantığının uygulanmasını sağlar.

MODÜL 3

KALİTE, GÜVENLİK VE OPERASYON

Production'a Hazır Uygulamalar

Validation • Exception Handling • Security • Swagger • Actuator

İçindekiler

- 9. Spring Validation API & Error Handling
- 10. Spring Security
- 11. Swagger/OpenAPI & Spring Actuator

9. Spring Validation API & Error Handling

Kullanıcıdan gelen verinin iş kurallarına uygunluğunu denetlemek, sağlam bir API'nin temel gereksinimidir. Spring, Jakarta Bean Validation (JSR-380) standardını Hibernate Validator implementasyonu ile birlikte kullanır.

9.1. Temel Validation Anotasyonları

Anotasyon	Açıklama
@NotNull	Değerin null olmamasını zorunlu kılar
@NotBlank	String değer boş veya sadece boşluk olmamasını zorunlu kılar
@NotEmpty	Koleksiyon/String'in boş olmamasını zorunlu kılar
@Size(min, max)	String veya koleksiyon uzunluğunu sınırlar
@Email	Geçerli e-posta formatı kontrolü yapar
@Min / @Max	Sayısal değer için alt/üst sınır belirler
@Pattern	Regex ile özel format doğrulaması yapar

9.2. @Valid ve @Validated Farkı

Anotasyon	Açıklama
@Valid (Jakarta)	Tekil nesne doğrulaması için kullanılır; iç içe (nested) nesnelerde de kademeli doğrulama tetikler
@Validated (Spring)	Sınıf seviyesinde kullanılabilir, Validation Group desteği sunar (örn. farklı senaryolarda farklı kural setleri)

```
public class OrderCreateDto {  
  
    @NotBlank(message = "Musteri adi bos olamaz")  
    private String customerName;  
  
    @Email(message = "Gecerli bir e-posta adresi giriniz")  
    private String email;  
  
    @Min(value = 1, message = "En az 1 urun secilmelidir")  
    private Integer stock;  
}  
  
@PostMapping  
public ResponseEntity<SiparisDto> olustur(@Valid @RequestBody OrderCreateDto request)  
{  
    // istek gecersizse MethodArgumentNotValidException otomatik firlatilir  
    return ResponseEntity.ok(orderService.create(request));  
}
```

9.3. Custom Validator Yazımı

Standart anotasyonların yetersiz kaldığı iş kurallarında (örneğin bir alanın belirli bir listedeki değerlerden biri olması), `@Constraint` anotasyonu ile özel doğrulayıcılar yazılabilir.

9.4. Global Exception Handling

Her controller'da tekrar eden try-catch blokları yazmak yerine, `@ControllerAdvice` ile uygulama genelinde merkezi bir istisna yönetim noktası oluşturulur.

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse>
validationHatasi(MethodArgumentNotValidException ex) {
        List<String> hatalar = ex.getBindingResult().getFieldErrors().stream()
            .map(FieldError::getDefaultMessage).toList();
        ErrorResponse hata = new ErrorResponse(LocalDate.now(), 400, "Validasyon
Hatasi", hatalar);
        return ResponseEntity.badRequest().body(hata);
    }

    @ExceptionHandler(SiparisBulunamadiException.class)
    public ResponseEntity<ErrorResponse> siparisBulunamadi(SiparisBulunamadiException
ex) {
        ErrorResponse hata = new ErrorResponse(LocalDate.now(), 404,
ex.getMessage(), null);
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(hata);
    }
}
```

9.5. Standart Hata Response Modeli

Tüm hata cevaplarının tutarlı bir yapıda dönmesi, istemci tarafında hata işlemeyi kolaylaştırır. Tipik bir hata modeli şu alanları içerir:

Alan	Açıklama
timestamp	Hatanın oluştuğu zaman damgası
status	HTTP durum kodu
message / errors	Hata mesajı veya alan bazlı hata listesi
path	Hatanın oluştuğu endpoint

İYİ UYGULAMA — En İyi Uygulama

İş kurallarına özel (`SiparisBulunamadiException` gibi) checked/unchecked istisna sınıfları tanımlamak, hem kodun okunabilirliğini artırır hem de `@ExceptionHandler` ile HTTP status kodlarının doğru biçimde eşlenmesini sağlar.

10. Spring Security

Spring Security, kimlik doğrulama (authentication) ve yetkilendirme (authorization) süreçlerini, bir dizi **filter chain** (filtre zinciri) üzerinden yöneten kapsamlı bir güvenlik framework'üdür.

10.1. Security Filter Chain Mantığı

Bir HTTP isteği uygulamaya ulaşmadan önce, `SecurityFilterChain` içinde tanımlı sıralı filtrelerden geçer: kimlik doğrulama filtresi, yetkilendirme filtresi, CSRF kontrolü, CORS kontrolü gibi. Her filtre kendi sorumluluğunu yerine getirir ve isteği bir sonrakine devreder.

10.2. Authentication vs Authorization

Kavram	Açıklama
Authentication (Kimlik Doğrulama)	"Siz kimsiniz?" sorusuna cevap verir — kullanıcı adı/şifre, token doğrulaması
Authorization (Yetkilendirme)	"Bu işlemi yapmaya yetkiniz var mı?" sorusuna cevap verir — rol/izin kontrolü

10.3. Temel Yapılandırma

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/**").permitAll()
                .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .sessionManagement(sm ->
                sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .addFilterBefore(jwtAuthFilter,
                UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

10.4. Şifreleme: PasswordEncoder

Kullanıcı şifreleri veritabanında **asla düz metin (plain text)** olarak tutulmamalıdır. `BCryptPasswordEncoder`, tek yönlü (one-way) ve salt içeren bir hash algoritması kullanarak şifreleri güvenli biçimde saklar.

10.5. JWT Tabanlı Authentication Mantığı

Stateless (durumsuz) REST API'lerde oturum bilgisi sunucu tarafında tutulmaz; bunun yerine kullanıcı giriş yaptığında imzalı bir **JWT (JSON Web Token)** üretilir. İstemci sonraki her istekte bu token'ı **Authorization: Bearer <token>** header'ı ile gönderir; sunucu token imzasını doğrulayarak kullanıcıyı tekrar veritabanına sormadan tanır.

- **Header:** Token türü ve imzalama algoritması bilgisi
- **Payload:** Kullanıcı kimliği, roller, geçerlilik süresi (exp) gibi claim'ler
- **Signature:** Sunucunun gizli anahtarıyla oluşturduğu imza — token'ın değiştirilmediğini garanti eder

10.6. Role-Based ve Method-Level Yetkilendirme

```
@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
    orderService.delete(id);
    return ResponseEntity.noContent().build();
}
```

10.7. CORS ve CSRF'ye Giriş

Kavram	Açıklama
CORS (Cross-Origin Resource Sharing)	Farklı origin'den (domain/port) gelen tarayıcı isteklerinin hangi koşullarda kabul edileceğini tanımlar
CSRF (Cross-Site Request Forgery)	Kullanıcının bilgisi dışında yetkili bir istek gönderilmesini önleyen koruma; stateless JWT tabanlı API'lerde genellikle devre dışı bırakılır, session tabanlı uygulamalarda aktif tutulmalıdır

11. Swagger/OpenAPI & Spring Actuator

Bir API'nin kendisi kadar, o API'nin nasıl kullanılacağını ve production ortamında sağlıklı çalışıp çalışmadığının izlenebilir olması da kritik önemdedir. Bu bölüm iki tamamlayıcı aracı ele alır: dokümantasyon için OpenAPI/Swagger, gözlemlenebilirlik için Spring Actuator.

11.1. springdoc-openapi Entegrasyonu

springdoc-openapi-starter-webmvc-ui bağımlılığı eklendiğinde, Spring Boot uygulamasındaki tüm `@RestController` endpoint'leri otomatik olarak taranır ve interaktif bir **Swagger UI** (/swagger-ui.html) ile OpenAPI JSON şeması (/v3/api-docs) üretilir — herhangi ek bir kod yazmaya gerek kalmadan.

11.2. Anotasyonlarla API Dokümantasyonunu Zenginleştirme

```
@Operation(summary = "Siparis detayini getirir", description = "ID ile tekil siparis sorgular")
@ApiResponses({
    @ApiResponse(responseCode = "200", description = "Basarili"),
    @ApiResponse(responseCode = "404", description = "Siparis bulunamadi")
})
@GetMapping("/{id}")
public ResponseEntity<OrderDto> getir(@PathVariable Long id) {
    return ResponseEntity.ok(orderService.getById(id));
}
```

11.3. Spring Actuator: Production İzlenebilirliği

spring-boot-starter-actuator bağımlılığı, uygulamanın çalışma zamanı durumunu HTTP endpoint'leri üzerinden dışarıya açar. Bu, özellikle Kubernetes gibi orkestrasyon araçlarının **liveness/readiness probe** mekanizmaları için doğrudan kullanılır.

Endpoint	Açıklama
/actuator/health	Uygulamanın ve bağlı olduğu servislerin (veritabanı, disk vb.) sağlık durumu
/actuator/info	Uygulama hakkında özel bilgiler (versiyon, build tarihi vb.)
/actuator/metrics	JVM bellek, CPU, HTTP istek sayıları gibi ölçüm verileri
/actuator/env	Aktif environment değişkenleri ve konfigürasyon kaynakları
/actuator/loggers	Çalışma zamanında log seviyelerini görüntüleme ve değiştirme

```
management:
  endpoints:
    web:
      exposure:
        include: health, info, metrics
  endpoint:
    health:
      show-details: when-authorized
```

11.4. Production Ortamında Actuator Güvenliği

DİKKAT — Dikkat

Actuator endpoint'leri, özellikle `/env` ve `/heapdump` gibi hassas bilgi içerenler, **asla kimlik doğrulamasız biçimde dışarıya açık bırakılmamalıdır**. Production'da yalnızca gerekli endpoint'ler `exposure.include` ile açılmalı ve Spring Security ile erişim kısıtlanmalıdır.

Swagger/OpenAPI ve Actuator birlikte, bir API'nin hem geliştirici ekipler hem de operasyon (DevOps) ekipleri tarafından güvenle kullanılmasını ve izlenmesini sağlayan tamamlayıcı iki production-readiness aracıdır.

Neominal Akademi

Spring Framework & Spring Boot Eğitimi | 2026 | v1.0

Tüm hakları saklıdır